

Principi di programmazione orientata agli oggetti
Istituzioni di programmazione orientata agli oggetti
a.a. 2024/25

Prova scritta dell'11 luglio 2025

Esercizio 1 (10 punti).

Sia data la classe `Node<T extends Comparable<T>>`:

```
class Node<T extends Comparable<T>> {
    private T elem;
    private Node<T> next;

    public Node(T elem, Node<T> next) {
        this.elem=elem;
        this.next=next; }
    public T getElem(){ return elem; }
    public void setElem(T elem) { this.elem= elem; }
    public Node<T> getNext(){ return next; }
}
```

`Node<T extends Comparable<T>>` implementa le liste concatenate i cui valori hanno come tipo una classe `T` generica. `T` implementa l'interfaccia `Comparable<T>` per poter confrontare i valori contenuti nei nodi tramite il metodo `compareTo()`. Inoltre, `null` denota la lista vuota.

Supponete di avere come parametri:

- una lista rappresentata dal nodo `p` di tipo `Node<T extends Comparable<T>>`, e che gli elementi di `p` siano tutti $\neq$ null (dunque potete invocare `compareTo()` su tali elementi).
- un valore `v` di tipo `T`.

Realizzate un metodo (implementato in modo ricorsivo):

```
public static <T extends Comparable<T>> Node<Integer>
    count(Node<T> p, T v)
```

che conta gli elementi uguali a `v` contenuti nella lista `p` e restituisce una lista formata da un solo elemento di tipo `Node<Integer>` che contiene il numero degli elementi uguali a `v`.

Esempi

Nel caso sia `T=Integer`, `count(p,v)` con `v=0` deve restituire:

- `[1] -> [0]`
- `[3, 4, 8] -> [0]`
- `[1, 0, 5, 0, 4] -> [2]`